Midterm Project Blueprint - The Hunter: Automated Email Phishing Defense Detection System

Course: ITAI 2376 - Deep Learning in AI
Institution: Houston City College
Date : March 31, 2026

Team Name: Team 1
Team Members:
• Chloe Tu
• Mattew Choo
• Franck Kolontchang

---

1. Problem Statement

Phishing is the number one entry point for data breaches, ransomware infections, and corporate account takeovers worldwide. The FBI's Internet Crime Complaint Center recorded over 298,000 phishing victims in 2024, with reported losses exceeding $18.7 million. The true number is far higher because most incidents go unreported. This is not a rare edge case. It is the most common and most costly form of cyberattack targeting people and organizations every single day.

The problem is not awareness. The problem is volume and speed. A corporate inbox can receive hundreds of emails per hour. Phishing emails are crafted specifically to look legitimate. They use urgency language, spoofed sender addresses, embedded redirect links, and credential-harvesting text designed to trick both humans and simple rule-based filters. By the time a suspicious email is manually reviewed, the damage is often already done.

The Hunter solves this by deploying a team of three autonomous AI agents that work together to evaluate every incoming email. Each agent perceives the information it is responsible for, reasons about it using its own judgment and tools, and takes action based on what it determines. The system scores phishing risk using a trained deep learning model, performs deeper investigation when confidence is low, delivers a concrete security verdict, and maintains a persistent memory of past threats. It does this continuously and automatically, without requiring human intervention for every email.

The organizations that benefit directly include corporate IT security teams, university systems, healthcare providers, financial institutions, and any email service provider whose users are targets of social engineering and credential-harvesting attacks.

---

2. Option Choice

Option B: Multi-Agent System

The Hunter is built as a multi-agent system because the problem has three fundamentally different jobs that require three different kinds of expertise and three different kinds of reasoning.

The first job is perception and structuring. Before any model can classify an email, someone has to make sense of the raw input. Raw email text contains HTML tags, encoding artifacts, obfuscated URLs, and inconsistent formatting. An agent whose only job is to perceive and normalize this input can focus entirely on doing that well, without being distracted by classification or policy decisions.

The second job is threat assessment. Once the email is structured, an agent needs to evaluate its risk. This requires running a deep learning model, interpreting the score, and making a judgment call about whether that score is confident enough to act on. If the score is ambiguous, the agent has to decide independently whether to investigate further. That is a reasoning task. No script tells the agent what to do. The agent uses its tools, reads the results, and decides.

The third job is response and memory. After a risk score is determined, an agent needs to decide what action protects the organization, execute that action, and record what happened so the system learns from it over time. This agent also reads the organization's threat history before it acts, because a repeat sender changes the calculus.

These three jobs cannot be collapsed into one agent without producing a confused, overloaded system that mixes perception, reasoning, and governance into a single context window. A multi-agent design separates them cleanly, mirrors how real Security Operations Center teams function, and produces a system where each agent is genuinely autonomous within its domain.

---

3. Agent Architecture

The Hunter uses three agents operating in a sequential pipeline. Each agent receives the prior agent's output as structured context, perceives what it needs to know, reasons using its available tools, and produces a structured output for the next agent. The key architectural property that makes this system truly agentic is that no agent is given a script. Each agent is given a goal and a set of tools. The decisions about which tools to call, in what order, and when the job is done are made entirely by the agent's own reasoning at runtime.

Agent 2 is the clearest example of this. It receives the cleaned email and its goal is to produce a risk score it is confident about. It has two tools available. Whether it needs one or both depends entirely on what the first tool returns. The agent reads the intermediate result, reasons about its own confidence, and decides. That decision is not in the Python code. It emerges from the model's judgment. This is the perceive-reason-act loop that defines agentic behavior.

Agent 3 demonstrates a second agentic property: memory-informed reasoning. Before issuing a verdict, Agent 3 reads the sender's threat history. That history was not in the original email. The agent is reasoning about information it discovered during the task, and that information can change its output. An agent that simply executes a function cannot do this.

System Architecture Diagram

THE HUNTER - Automated Email Phishing Defense Detection System
CrewAI | Three Autonomous Agents | Sequential Pipeline

[RAW EMAIL INPUT]

|
v

AGENT 1: Email Ingest Specialist

PERCEIVES: Raw email body text

GOAL: Prepare a clean, structured representation of this email that gives the risk analyst everything needed to make an accurate threat assessment.

AVAILABLE TOOL:
email_ingest_tool

- Strips HTML tags
- Normalizes URLs to "url" token
- Extracts sender fingerprint

- Computes 5 phishing signal features: char_count, word_count, url_count, exclaim_count, urgent_keyword_count

ACTS: Calls email_ingest_tool, verifies output is complete and well-formed

OUTPUT: JSON (clean_text, features, feature_names, sender_hint)

|
v

structured email context passed forward

|
v

AGENT 2: Phishing Risk Analyst

PERCEIVES: Clean email text + 5 features from Agent 1

GOAL: Determine whether this email is a phishing attempt with enough confidence to justify a security action. Use whatever tools are necessary to be confident.

AVAILABLE TOOLS:
phishing_classifier_tool

- BiLSTM text model (65% weight)
- Logistic Regression (35% weight)
- PR-curve optimal threshold
- Returns risk_score + confidence flag

deep_analysis_tool

- 6 regex phishing archetype patterns
- 4 structural red-flag checks
- Adjusts score by up to +0.3
- Returns adjusted_risk_score

AGENT REASONS:
"Is my current confidence sufficient to pass a score forward? Or do I need to investigate further?"
| -- Confident (score < 0.35 or > 0.75) Agent passes score forward
|
| -- Uncertain (score 0.35 to 0.75) Agent calls deep_analysis_tool Agent re-evaluates, then passes adjusted score forward

OUTPUT: JSON (risk_score or adjusted_risk_score threshold_used, sender_hint)

|
v

risk assessment context passed forward

|

v

AGENT 3: SOC Orchestrator

PERCEIVES: Risk score from Agent 2 + sender threat history from SQLite

GOAL: Issue the security response that best protects the organization, and maintain the institutional threat memory so future agents can reason about repeat offenders.

AVAILABLE TOOLS:
defense_policy_tool

- 4-tier policy on calibrated threshold:
  - score >= 0.90 --> QUARANTINE
  - score >= 0.70 --> FLAG + WARNING
  - score >= t* --> ALLOW AND LOG
  - score < t* --> ALLOW
  - (t* = PR-curve optimal threshold)

threat_memory_tool

- Reads sender history from SQLite
- Writes new verdict to SQLite
- Returns past_incidents + repeat flag

AGENT REASONS:
"What does the risk score say? What does this sender's history say? What is the right action for the org?"

ACTS: Issues verdict, logs to memory, flags repeat offenders in final output

OUTPUT: JSON (action, explanation, risk_score, past_incidents, repeat_offender, memory_summary)

|
v

[FINAL VERDICT + THREAT LOG ENTRY]

THE HUNTER

Automated Email Phishing Defense Detection System

CrewAI | 3 Autonomous Goal-Driven Agents | Sequential Pipeline | ITAI 2376 Spring 2026

INPUT

Raw Email Text + Sender Info

AGENT 1 — Email Ingest Specialist

PERCEIVE › REASON › ACT

PERCEIVES

Raw email body text

HTML tags, encoded chars, obfuscated URLs, noise

GOAL

Prepare a clean structured representation so the Risk Analyst can accurately assess the threat.

TOOL: email_ingest_tool

- › Strip HTML tags and encoding artifacts
- › Normalize all URLs to "url" token
- › Extract sender fingerprint
- › Compute 5 features: char_count, word_count, url_count, exclaim_count, urgent_kw_count

OUTPUT JSON: clean_text | features[5] | feature_names | sender_hint

structured email context

AGENT 2 — Phishing Risk Analyst

PERCEIVE › REASON › ACT

PERCEIVES

Clean text + 5 features from Agent 1 JSON output

GOAL

Determine phishing risk with enough confidence to justify a security action. Use whatever tools needed.

TOOL A: phishing_classifier_tool

- › BiLSTM text model (65% weight)
- › Logistic Regression (35% weight)
- › PR-curve calibrated threshold
- › Returns risk_score + confidence

Confident?

score < 0.35 or > 0.75 ?

YES — pass score forward

NO — investigate further

TOOL B: deep_analysis_tool [CONDITIONAL — Agent 2 decides to call this when confidence is insufficient]

6 regex phishing archetypes | 4 structural red-flag checks | score boost up to +0.34 → adjusted_risk_score

"My score is ambiguous. I need to investigate further before passing a verdict forward."

OUTPUT JSON: risk_score or adjusted_risk_score | threshold_used | sender_hint

risk assessment context

AGENT 3 — SOC Orchestrator

PERCEIVE › REASON › ACT

PERCEIVES

Risk score from Agent 2

Sender threat history from SQLite long-term memory (threat_memory.db)

GOAL

Issue the security response that best protects the org. Maintain threat memory so future agents can reason about repeat offenders.

TOOL A: defense_policy_tool

- › score ≥ 0.90 → QUARANTINE
- › score ≥ 0.70 → FLAG + WARNING
- › score ≥ t* → ALLOW AND LOG
- › score < t* → ALLOW

t* = PR-curve optimal threshold

TOOL B: threat_memory_tool

- › Read last 5 sender records
- › Write new verdict to SQLite
- › Flag REPEAT OFFENDER

Persists across sessions via Google Drive mount

OUTPUT JSON: action | explanation | risk_score | past_incidents | repeat_offender | memory_summary

FINAL SECURITY VERDICT + THREAT LOG ENTRY

QUARANTINE | **FLAG + WARNING** | **ALLOW AND LOG** | **ALLOW**

DEEP LEARNING CONNECTIONS

- Module 04: BiLSTM (RNN Family)
- Module 05: Transformers / LLaMA 3
- Module 02: Embeddings
- Framework: CrewAI (Python)

Primary Dataset: Alam Kaggle 28,747 labeled emails | Secondary: Cratchley 500k+ emails

ITAI 2376 Deep Learning in Artificial Intelligence | Houston City College | Spring 2026

Why This Is Agentic, Not Just a Pipeline

A pipeline executes a fixed sequence of functions with no reasoning between steps. An agentic system gives each agent a goal and the capability to choose how to reach it. The difference is visible in two places in this design.

Agent 2 is not told to call the deep_analysis_tool. It is told to produce a confident risk score. If the classifier output is ambiguous, the agent determines that one tool is not enough and calls the second one. That is a decision the agent makes based on what it perceives. The code does not make that decision.

Agent 3 reads threat history before issuing its verdict. That history was not in the original email. The agent discovers it, reasons about what it means (is this a first-time sender or a repeat offender?), and produces an output shaped by that reasoning. The verdict the agent issues is not predetermined.

---

4. Deep Learning Connection

Module 04: Recurrent Neural Networks / Bidirectional LSTM

The primary classification model powering the phishing_classifier_tool is a Bidirectional Long Short-Term Memory (BiLSTM) network. This is a direct application of the Recurrent Neural Network family covered in Module 04.

Phishing detection is a sequence classification problem. The meaning of individual words like "verify," "suspended," or "click" depends heavily on the words around them. The phrase "please verify your identity to avoid account suspension" is a phishing signal. The phrase "we were unable to verify the package identity at this station" is not. A feedforward network cannot distinguish these because it treats each token independently. The BiLSTM reads the email as a sequence in both directions simultaneously: the forward LSTM from first token to last, and the backward LSTM from last to first. Their hidden states are concatenated at every position, giving the model full bidirectional context at each word.

Stacking two BiLSTM layers (128 units into 64 units) builds hierarchical representations. The first layer captures local phishing patterns like urgency phrases. The second layer captures longer-range combinations like a URL following a credential request. This is exactly the architecture Module 04 prepares students to build and understand. The model is trained on 28,747 real emails from the Alam Kaggle dataset with SMOTE oversampling,

class_weight="balanced," SpatialDropout1D(0.3), Dropout(0.4), EarlyStopping, and ReduceLROnPlateau.

Module 05: Transformers as Agent Reasoning Engines

Each of the three CrewAI agents is powered by Meta's LLaMA 3 8B model, served through the Groq inference API. LLaMA 3 is a Large Language Model built on the Transformer architecture covered in Module 05. The self-attention mechanism is what allows each agent to read a structured JSON payload, reason about what it means, decide which tool to call, construct the correct arguments, evaluate the result, and determine when its job is done.

This is the architectural division of labor that makes the system work. The BiLSTM does the deep sequence classification that Transformers are not optimized for at training scale. The Transformer does the reasoning, judgment, and tool orchestration that the BiLSTM cannot do at all. Both models are essential. Neither could replace the other. This is also what makes the system agentic: the Transformer is not executing a function, it is thinking about what function to execute and why.

Module 02: Representation Learning and Embeddings

The BiLSTM input passes through a trainable Embedding(20000, 128) layer before reaching the recurrent layers. This is a direct application of representation learning from Module 02. Rather than feeding raw integer token indices into the LSTM, the embedding layer maps each vocabulary token to a 128-dimensional dense vector. These vectors are learned during training and encode semantic relationships. Tokens that appear in similar phishing contexts cluster together in the embedding space. This means the BiLSTM receives richer, more generalizable signal from its very first layer rather than treating "verify" and "confirm" as completely unrelated tokens.

---

5. Agent Framework

Framework: CrewAI (Python)

CrewAI was chosen as the agent framework for this project for three specific reasons, and it has already been prototyped during the research phase of this blueprint.

Reason 1: Role-based agent design enforces separation of concerns. In CrewAI, every agent has a declared role, goal, and backstory that is passed directly into its system prompt. This means Agent 2 is constitutionally a Phishing Risk Analyst and will not drift into issuing policy verdicts

(Agent 3's job) or preprocessing text (Agent 1's job). This boundary enforcement reduces the rate of LLM hallucinations caused by context overload, which is one of the most common failure modes in multi-agent systems.

Reason 2: Goal-driven tasks rather than scripted instructions. CrewAI tasks are written as goals and expected outputs, not step-by-step instructions. This is what separates CrewAI from simpler chaining frameworks. Agent 2's task says "produce a confident phishing risk assessment using your available tools," not "call tool A then call tool B if condition X." The agent reasons about how to accomplish the goal. This is the design pattern that produces genuine agentic behavior.

Reason 3: Native Pydantic tool schemas prevent runtime type errors. CrewAI's BaseTool class enforces typed Pydantic input schemas on every tool before execution. All tool inputs are declared as strings. This eliminates the class of bug where the LLM passes a Python dictionary to a field that expects a serialized JSON string, a real failure mode that crashed an earlier version of this system on borderline test cases.

---

6. Tools and Data

The Five Agent Tools

Tool	Agent	What It Does
email_ingest_tool	Agent 1	Strips HTML, normalizes URLs to "url" token, extracts sender fingerprint, computes 5 numeric phishing signal features
phishing_classifier_tool	Agent 2	Runs the stacked BiLSTM (65% weight) and Logistic Regression (35% weight) ensemble, returns weighted risk_score calibrated to PR-curve optimal threshold
deep_analysis_tool	Agent 2	Matches 6 regex phishing archetype patterns and 4 structural red-flags, computes a score boost of up to +0.34, returns adjusted_risk_score
defense_policy_tool	Agent 3	Maps the final risk score to one of four security actions (QUARANTINE, FLAG WITH WARNING, ALLOW

		AND LOG, ALLOW) using the PR-curve optimal threshold
threat_memory_tool	Agent 3	Reads the sender's last 5 verdicts from SQLite, writes the new verdict, flags REPEAT OFFENDER if a prior FLAG or QUARANTINE is on record

Datasets

Primary: Alam Phishing Email Dataset Source: Kaggle (naserabdullahalam/phishing-email-dataset) Size: 28,747 labeled raw email bodies Role: Training data for both the BiLSTM text model and the 5-feature Logistic Regression model

The Alam dataset is the primary training source because it contains the same kind of input the system receives at inference time: raw email body text. A model trained on raw text can classify raw text at deployment without any additional preprocessing pipeline. The five numeric features extracted during training are the exact same five features extracted by email_ingest_tool at runtime. This alignment between training and inference is what makes the classifier reliable.

Secondary: Cratchley Email Phishing Dataset Source: Kaggle (ethancratchley/email-phishing-dataset) Size: 500,000+ emails with pre-engineered header and metadata features Role: Standalone analysis only, demonstrating that phishing signals exist in structural metadata beyond the email body

The Cratchley dataset is evaluated in the notebook using a standalone Random Forest model to show the breadth of phishing signal categories. It is not wired into the live agent tools because its engineered features (domain age, DNS lookup results, header structure) require a full email parsing and DNS resolution pipeline that is not available from raw email body text at inference time.

Infrastructure

- Groq API (free tier): LLaMA 3 8B inference endpoint for agent reasoning
- TensorFlow and Keras: BiLSTM training and inference
- scikit-learn and imbalanced-learn: Logistic Regression, StandardScaler, SMOTE oversampling
- SQLite: Persistent threat memory database, no server required, runs in Google Colab
- Google Colab: Training and demo environment

---

7. Build Plan

The timeline below covers the full period from midterm submission through the final project deadline.

Week	Dates	Goal
Week 1	Mar 27 - Apr 2	Submit blueprint. Set up Colab environment with CrewAI, TensorFlow, Groq, imbalanced-learn, and Kaggle API. Download both datasets. Confirm data loads cleanly, column names are correct, and label classes are properly mapped.
Week 2	Apr 3 - Apr 9	Complete exploratory data analysis: class distributions, text length distributions, feature correlation. Implement the extract_features() function with the same 5 columns used at inference. Train the Logistic Regression with SMOTE. Save scaler and confirm it reloads correctly.
Week 3	Apr 10 - Apr 16	Build and train the stacked BiLSTM. Generate training history plots for loss, accuracy, precision, and recall per epoch. Compute the PR-curve optimal threshold. Save all models to disk. Verify the full reload pipeline works end to end.
Week 4	Apr 17 - Apr 23	Build all five CrewAI tools with Pydantic schemas. Unit test each tool independently with direct Python calls before connecting any agents. Write goal-oriented task descriptions. Build all three agents, configure LLaMA 3 via Groq, and verify the API connection.
Week 5	Apr 24 - Apr 30	Wire the full pipeline with run_hunter(). Run four demo emails: a clear phishing email, a clean legitimate email, a borderline email to confirm Agent 2 escalates autonomously to deep_analysis_tool, and a repeat-sender email to confirm Agent 3 surfaces the repeat offender flag from memory. Debug and polish.
Week 6	May 1 - May 3	Record the 8-minute demo video. Final notebook cleanup, add README with setup instructions, and package the full submission. Submit Final Agent, Write-Up, and Demo.

8. Anticipated Challenges

Challenge 1: Keeping Agent Tasks Goal-Oriented, Not Scripted

The biggest threat to agentic behavior in this system is over-specifying the task descriptions. If a task tells the agent exactly which tool to call and when, the agent becomes a function executor rather than a reasoner. The risk is highest for Agent 2, where the temptation is to write "if score is between 0.35 and 0.75, call deep_analysis_tool."

The plan is to write all task descriptions as goals and expected outputs only. Agent 2's task will say "produce a phishing risk score you are confident enough to act on, using whatever tools you need." The agent decides the rest. This will be validated during Week 4 by testing the agent on borderline emails and confirming it escalates on its own without any conditional logic in the Python code.

Challenge 2: Borderline Email Misclassification

Emails in the ambiguous risk range are the hardest cases for any classifier. A hardcoded 0.5 threshold draws the decision boundary arbitrarily, causing phishing emails with moderate signals to slip through as ALLOW verdicts.

The plan is to compute the optimal threshold from the Precision-Recall curve on the held-out test set during training. That value is saved to disk and loaded into the phishing_classifier_tool at runtime. The deep_analysis_tool provides a second signal specifically for ambiguous cases by applying regex pattern matching across six known phishing archetypes. Combined, these two mechanisms reduce the rate of dangerous misclassifications in the gray zone.

Challenge 3: Class Imbalance in Real-World Data

Real email inboxes are not balanced. Legitimate emails vastly outnumber phishing emails. A model trained on an imbalanced dataset without correction learns to predict "Safe" for everything and still achieves high raw accuracy, which is useless for a security system. An earlier version of this project trained a Random Forest without imbalance correction and got 0.34 precision and 0.31 recall on the phishing class.

The plan is to apply SMOTE oversampling to the training set before fitting the Logistic Regression. The class_weight="balanced" parameter is passed to both the Logistic Regression and the BiLSTM's fit() call. EarlyStopping monitors val_loss rather than accuracy to prevent the model from collapsing to majority-class prediction.

Challenge 4: SQLite Persistence Across Colab Sessions

Google Colab does not persist files between runtime sessions. A threat memory database written during one session disappears when the runtime resets, which breaks the repeat-offender logic.

The plan is to include a clearly commented code block in the notebook to mount Google Drive and store threat_memory.db in Drive rather than the local Colab filesystem. The _init_db() startup function uses CREATE TABLE IF NOT EXISTS, so the schema is created fresh if the database file is missing and preserved intact if it already exists from a prior session.